



**TECNOLÓGICO
NACIONAL DE MÉXICO**



TOPICOS DE IA

TEMA:

Optimización de rutas de distribución utilizando Recocido
Simulado

Alumnos:

Alfonso Lara Torres

Jose Enrique Felix Esparragoza

Profesor:

Zuriel Dathan Mora Felix

1. Introducción

En el entorno empresarial actual, la eficiencia en la logística de distribución representa un factor determinante para la competitividad y sostenibilidad de las organizaciones. En particular, las empresas dedicadas a la distribución de productos de abarrotes enfrentan el reto constante de optimizar sus rutas de entrega para reducir costos operativos, mejorar los tiempos de distribución y garantizar un servicio eficiente a sus clientes. Este desafío se intensifica cuando el número de puntos de entrega y las variables del entorno aumentan, generando un problema complejo conocido como Problema de Rutas de Vehículos.

El problema de ruteo de vehículos (PRV) puede describirse como el problema de diseñar rutas óptimas de entrega o recolección desde uno o varios depósitos hacia un conjunto de ciudades o clientes geográficamente dispersos, sujeto a diversas restricciones. Este problema desempeña un papel central en la distribución física y la logística (Laporte, 1992).

El presente trabajo aborda la optimización de las rutas de distribución de productos de abarrotes desde los centros de distribución hacia las distintas sucursales de una cadena de restaurantes ubicada en Culiacán, Sinaloa. El cual está conformado 90 tiendas y 10 centros de distribución.

Ante este panorama, se propone la aplicación del método heurístico de Recocido Simulado como una alternativa eficaz para encontrar soluciones de alta calidad en un tiempo razonable.

Las heurísticas son criterios, métodos o principios que permiten decidir, entre varias alternativas, cuál promete ser la más efectiva para alcanzar un objetivo. Representan un compromiso entre la simplicidad y la capacidad de discriminar correctamente entre buenas y malas elecciones (Pearl, 1984, pág. 3).

El uso de Recocido Simulado en la optimización de rutas permite modelar el proceso de búsqueda de soluciones de manera inspirada en el enfriamiento de metales, explorando diversas configuraciones posibles y evitando quedar atrapado en óptimos locales. De esta manera, se busca mejorar la eficiencia de las rutas de distribución, reducir los costos operativos y optimizar el uso de recursos logísticos dentro de la empresa.

2. Objetivo General

Desarrollar una estrategia de optimización de las rutas de distribución de productos de abarrotes desde los centros de distribución hasta las sucursales de una cadena de restaurantes en Culiacán, Sinaloa, mediante la aplicación del método de búsqueda heurística de Recocido Simulado, con el fin de reducir los costos operativos, mejorar la eficiencia logística y generar soluciones cercanas al óptimo para la planificación de entregas.

3. Objetivos Específicos

- Procesar y modelar los datos de distribución, incluyendo sucursales, centros de distribución, distancias y costos de combustible, para representar correctamente el problema logístico.

- Generar y evaluar una solución inicial de rutas, calculando costos, distancias y consumo de gasolina como punto de partida para la optimización.
- Aplicar el algoritmo de Recocido Simulado, generando vecinos y optimizando iterativamente las rutas para minimizar costos, distancias y consumo de combustible.
- Analizar y visualizar los resultados, comparando la solución inicial con la optimizada y mostrando las rutas finales en un mapa interactivo para facilitar la interpretación.

4. Justificación

Resolver el problema de optimización de rutas mediante un método de búsqueda heurístico es relevante debido a la complejidad inherente de la logística de distribución. Este tipo de problemas involucra múltiples sucursales y combinaciones posibles, lo que hace inviable una búsqueda exhaustiva de soluciones óptimas en tiempo razonable. Como señala KeepCoding (2023, párr. 8), “las empresas de transporte utilizan algoritmos heurísticos para encontrar las rutas más rápidas o eficientes, optimizando el tiempo de entrega”.

La búsqueda heurística, utilizada durante siglos como herramienta eficaz, aplica principios matemáticos, informáticos y de inteligencia artificial para obtener soluciones óptimas o cercanas al óptimo en problemas específicos (Complexica, s.f.). Su aplicación impacta directamente en la toma de decisiones logísticas, ya que permite planificar rutas eficientes, reducir costos operativos, disminuir distancias recorridas y consumo de combustible, y adaptar la operación a cambios en la demanda.

5. Alcance

El proyecto se centra en optimizar las rutas de distribución de productos de abarrotes desde los centros de distribución hacia las sucursales de una cadena de restaurantes en Culiacán, Sinaloa, utilizando el método heurístico de Recocido Simulado. Para ello, se consideran aspectos como la asignación inicial de tiendas a los centros de distribución, la minimización del costo ponderado total (distancia recorrida y consumo de combustible) y la generación de rutas más eficientes mediante operaciones de intercambio (swap) y reasignación de nodos, que permiten explorar distintas configuraciones de entrega. Los resultados se presentan mediante un mapa interactivo que facilita la interpretación de las rutas optimizadas.

No se incluirán factores externos como el tráfico en tiempo real, demanda de cada tienda o sucursales prioritarias.

Como resultado, se espera obtener rutas optimizadas que permitan disminuir los costos de transporte, acortar las distancias recorridas y reducir el consumo de combustible, proporcionando además una visualización clara que facilite la toma de decisiones logísticas.

6. Desarrollo

6.1 Descripción del Problema

El presente proyecto aborda el desafío logístico que enfrenta una cadena de restaurantes en Culiacán, Sinaloa, en la distribución de productos de abarrotes desde sus centros de

distribución hacia sus múltiples sucursales. La red logística de la empresa está compuesta por 10 centros de distribución encargados de abastecer a un total de 90 tiendas geográficamente dispersas en la ciudad.

Este escenario configura un Problema de Ruteo de Vehículos (PRV), que consiste en diseñar un conjunto de rutas de entrega óptimas para minimizar los costos operativos y mejorar la eficiencia (Laporte, 1992). La complejidad del problema radica en el elevado número de combinaciones de rutas posibles, lo que hace inviable encontrar una solución óptima mediante una búsqueda exhaustiva en un tiempo razonable. El objetivo principal es, por tanto, minimizar un costo ponderado total que considera dos factores clave: la distancia total recorrida y el consumo de combustible asociado a dichas rutas.

Para enfocar la solución, el alcance del problema se limita a la optimización estática de las rutas. El objetivo principal es minimizar una función de costo ponderado que, tal como se implementa en el programa, integra tanto la distancia total recorrida como el consumo de combustible. Por consiguiente, el modelo no considera variables dinámicas externas como el tráfico en tiempo real, la demanda específica de cada sucursal o la priorización de entregas, centrándose exclusivamente en encontrar la configuración de rutas más eficiente con base en los datos geográficos y de costos provistos.

6.2 Selección y Explicación de Recocido Simulado

Para abordar el problema de optimización de rutas de distribución planteado en este proyecto, se seleccionó el algoritmo de Recocido Simulado como estrategia de solución. Esta elección se fundamenta en diversos criterios que se detallan a continuación.

El Recocido Simulado es un algoritmo probabilístico de optimización inspirado en el proceso de recocido en metalurgia, en el que un material se calienta y luego se enfría lentamente para modificar sus propiedades físicas. El algoritmo imita este proceso para buscar una aproximación al óptimo global de una función dada. Una de sus características distintivas es que, además de aceptar soluciones que mejoran la función, también puede aceptar soluciones 'peores' con cierta probabilidad, lo cual le permite escapar de óptimos locales (Kirkpatrick, Gelatt, & Vecchi, 1983).

El problema de ruteo de vehículos que se aborda pertenece a la clase de problemas NP-difíciles (Lenstra & Rinnooy Kan, 2006), lo que significa que no existe un algoritmo de tiempo polinomial que garantice encontrar la solución óptima. Esta complejidad justifica el uso de métodos metaheurísticos que pueden encontrar soluciones de alta calidad en tiempo razonable.

El Recocido Simulado se distingue por su mecanismo de aceptación probabilística de soluciones peores mediante la función $P(\text{aceptar}) = \exp(-\Delta/T)$, donde Δ es el cambio en el costo y T es la temperatura. Esta característica permite explorar regiones del espacio de soluciones que métodos greedy ignorarían y evitar óptimos locales.

En términos de flexibilidad, el Recocido Simulado (SA) ofrece ventajas significativas. Se destaca por ser un algoritmo robusto y de implementación directa, al no requerir estructuras de datos complejas (Osman, 1993). Además, posee una gran capacidad para hibridarse

con otras técnicas como la búsqueda local y la post-optimización, potenciando así la calidad de las soluciones finales (Thangiah, Osman, & Sun, 2014). La literatura científica respalda consistentemente su efectividad, demostrando que el SA puede generar soluciones de alta calidad para problemas de ruteo en tiempos computacionales razonables.

Considerando estas características, el Recocido Simulado se consolida como una estrategia idónea para este proyecto. Su capacidad para abordar la complejidad NP-difícil del problema, junto con su mecanismo probabilístico para evadir óptimos locales, lo posicionan como un método robusto. Adicionalmente, su probada efectividad, flexibilidad y relativa simplicidad de implementación justifican plenamente su selección como el enfoque de optimización a seguir.

6.3 Implementación Computacional

La implementación del proyecto se realizó en Python, debido a su ecosistema de bibliotecas especializadas para el manejo de datos y la visualización:

- Pandas y NumPy: Fueron la base para la manipulación de datos. Pandas se utilizó para cargar los archivos .csv en DataFrames, mientras que NumPy permitió convertir esta información en matrices numéricas para realizar cálculos de distancia y costo de manera eficiente dentro del algoritmo.
- Matplotlib: Se empleó para generar las gráficas que analizan el rendimiento del algoritmo. Estas visualizaciones fueron cruciales para observar la convergencia del modelo al mostrar la evolución del costo, la distancia y el combustible a lo largo de las iteraciones.
- Folium: Fue la herramienta clave para la visualización geográfica de la solución final. Permitted generar un mapa interactivo en HTML que representa las rutas optimizadas, los centros de distribución y las tiendas, facilitando enormemente la interpretación de los resultados.

Una vez establecidas las herramientas para la gestión de datos y la visualización de resultados, el núcleo del algoritmo de Recocido Simulado se implementó de manera modular. El proceso se estructuró en una serie de funciones interconectadas, cada una con una responsabilidad específica dentro de la lógica del programa.

Solución inicial:

Para establecer un punto de partida para el algoritmo de optimización, se implementó la función `solucion_inicial`, la cual genera un conjunto de rutas base utilizando una heurística constructiva de tipo *greedy*, específicamente el algoritmo del vecino más cercano. Esta solución inicial asigna a cada tienda con su centro de distribución más cercano en distancia.

```
def solucion_inicial(df_tiendas, matriz_distancias):
    n_cedis = len(df_tiendas[df_tiendas["Tipo"] == "Centro de
Distribución"])
    rutas = [[] for _ in range(n_cedis)]
```

```

for i in range(len(df_tiendas)):
    tipo = df_tiendas.loc[i, "Tipo"]
    if tipo == "Tienda":
        distancias_a_cedis = matriz_distancias[i, :n_cedis]
        idx_min = np.argmin(distancias_a_cedis)
        rutas[idx_min].append(i)
return rutas

```

Costo de una ruta específica:

La función `costo_ruta` sirve como la función de evaluación para una ruta individual. Calcula el costo total de un recorrido cerrado que parte de un CEDIS, visita secuencialmente las tiendas asignadas y regresa al origen. Para ello, suma los valores de distancia y combustible entre paradas y aplica una función de costo ponderado, definida como $\text{costo} = (0.3 \cdot \text{distancia}) + (0.7 \cdot \text{gasolina})$, la cual prioriza la reducción del consumo de combustible como factor principal para la optimización.

```

def costo_ruta(ruta, id_cedis, matriz_distancias, matriz_combustible):
    costo_distancia = 0
    costo_gasolina = 0
    actual = id_cedis
    alpha=0.3
    beta=0.7
    for nodo in ruta:
        costo_distancia += matriz_distancias[actual][nodo]
        costo_gasolina += matriz_combustible[actual][nodo]
        actual = nodo
    costo_distancia += matriz_distancias[actual][id_cedis]
    costo_gasolina += matriz_combustible[actual][id_cedis]
    costo_ponderado = (alpha * costo_distancia) + (beta * costo_gasolina)
    return costo_ponderado, costo_distancia, costo_gasolina

```

Costo total:

La función `costo_total` actúa como un agregador para evaluar la calidad global de una solución completa. Esta itera sobre el conjunto de todas las rutas y, utilizando la función `costo_ruta` para cada una, realiza una suma acumulativa de sus costos ponderados, distancias y consumos de gasolina. El valor ponderado resultante es la métrica que el algoritmo busca minimizar.

```

def costo_total(rutas, matriz_distancias, matriz_combustible):
    total_ponderado = 0
    total_distancia = 0
    total_gasolina = 0
    for id_cedis, ruta in enumerate(rutas):
        costo_p, distancia, gasolina = costo_ruta(ruta, id_cedis,
matriz_distancias, matriz_combustible)

```

```

        total_ponderado += costo_p
        total_distancia += distancia
        total_gasolina += gasolina
    return total_ponderado, total_distancia, total_gasolina

```

Generación de nuevos estados/vecinos:

La función `generar_vecino` es la responsable de la exploración del espacio de soluciones, creando una nueva configuración de rutas a partir de la actual. Para ello, implementa *swap*, que consiste en seleccionar aleatoriamente dos rutas y una tienda dentro de cada una para intercambiarlas de posición. La función también contempla casos especiales, como mover una tienda a una ruta vacía si esta es seleccionada.

```

def generar_vecino(rutas):
    rutas_vecinas = [list(r) for r in rutas]
    m = len(rutas_vecinas)
    r1 = random.randint(0, m - 1)
    posibles_rutas = [i for i in range(m) if len(rutas_vecinas[i]) > 0 and i
!= r1]
    if not posibles_rutas:
        return rutas_vecinas
    r2 = random.choice(posibles_rutas)
    # Caso: la ruta r1 está vacía
    if len(rutas_vecinas[r1]) == 0:
        nodo = rutas_vecinas[r2].pop(random.randint(0,
len(rutas_vecinas[r2]) - 1))
        rutas_vecinas[r1].append(nodo)
        return rutas_vecinas
    # Caso general: intercambio entre rutas
    posibles_rutas = [i for i in range(m) if len(rutas_vecinas[i]) > 0]
    r2 = random.choice(posibles_rutas)
    idxNodo1 = random.randint(0, len(rutas_vecinas[r1]) - 1)
    nodo1 = rutas_vecinas[r1][idxNodo1]
    idxNodo2 = random.randint(0, len(rutas_vecinas[r2]) - 1)
    while r1 == r2 and idxNodo1 == idxNodo2:
        idxNodo2 = random.randint(0, len(rutas_vecinas[r2]) - 1)
    nodo2 = rutas_vecinas[r2][idxNodo2]
    rutas_vecinas[r1][idxNodo1] = nodo2
    rutas_vecinas[r2][idxNodo2] = nodo1
    return rutas_vecinas

```

Núcleo del Algoritmo de Recocido Simulado

La función `optimizacion_recocido_simulado` orquesta todo el proceso de optimización, integrando las funciones auxiliares en un ciclo iterativo. El proceso comienza con la solución generada por `solucion_inicial` y una temperatura alta (t_0). A partir de ahí, ejecuta un bucle

principal que se detiene solo cuando la temperatura desciende por debajo de un umbral final (tf). Dentro de este, un bucle anidado explora el espacio de soluciones a una temperatura constante:

1. Generación: Se crea una solución vecina mediante la función generar_vecino.
2. Evaluación: Se calcula el costo de esta nueva solución con costo_total.
3. Decisión: Si la solución vecina tiene un costo menor, se acepta incondicionalmente. Si tiene un costo mayor, puede ser aceptada con una probabilidad de $e^{\frac{-\Delta}{T}}$, donde Δ es el incremento del costo y T es la temperatura actual.
4. Enfriamiento: Una vez completadas las iteraciones para una temperatura dada, esta se reduce multiplicándola por un factor de enfriamiento.

Durante todo el ciclo, la función mantiene un registro de la mejor solución global encontrada hasta el momento. Al finalizar, devuelve esta solución óptima junto con su costo, distancia, consumo de combustible y los datos históricos del proceso para su posterior análisis y visualización.

Hemos omitido el código de esta función principal para centrar la atención en su flujo, debido a que resulta más ilustrativo para comprender el proceso de optimización que la implementación específica.

6.4 Resultados Obtenidos

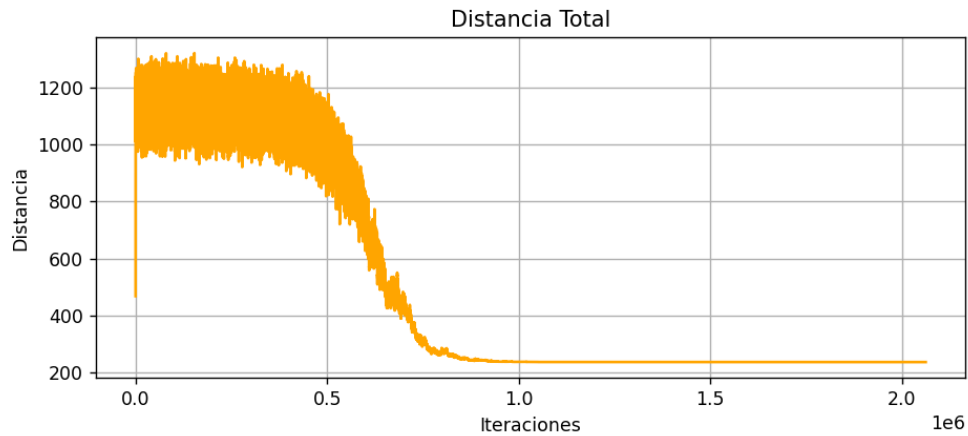
A continuación, se presentan los resultados obtenidos:

- mejor_costo_pond = 95.87
- mejor_distancia = 236.70
- mejor_gasolina = 35.51

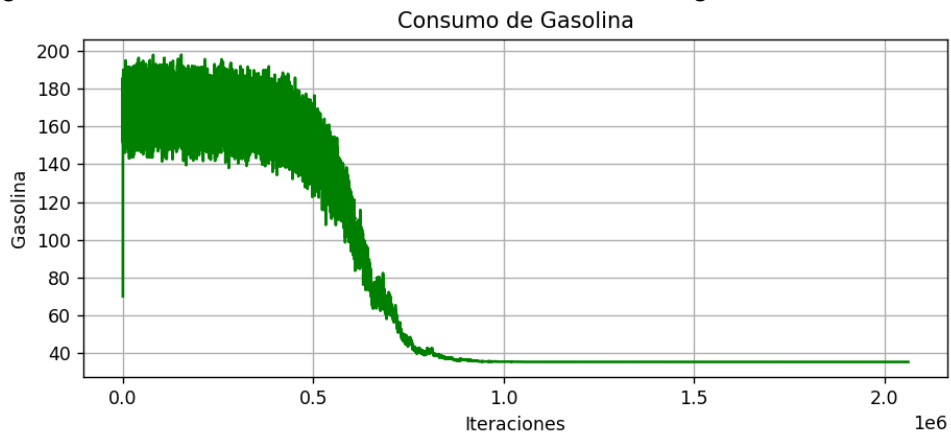
Tabla de rutas:

Ruta	Costo Ponderado	Distancia	Combustible
0, 63, 59, 64, 73, 33, 0	6.25	15.44	2.32
1, 70, 37, 85, 71, 18, 32, 1	7.38	18.23	2.73
2, 35, 97, 17, 26, 2	3.96	9.78	1.47
3, 60, 99, 14, 76, 57, 40, 94, 65, 34, 16, 46, 41, 68, 66, 91, 92, 12, 87, 44, 93, 61, 3	21.27	52.51	7.88
4, 75, 20, 74, 79, 22, 62, 67, 50, 88, 80, 95, 4	9.37	23.13	3.47
5, 96, 24, 42, 19, 27, 51, 77, 25, 5	7.37	18.21	2.73
6, 86, 31, 89, 53, 72, 48, 84, 52, 36, 11, 43, 58, 39, 54, 98, 82, 69, 6	18.36	45.32	6.80
7, 47, 23, 7	3.30	8.14	1.22

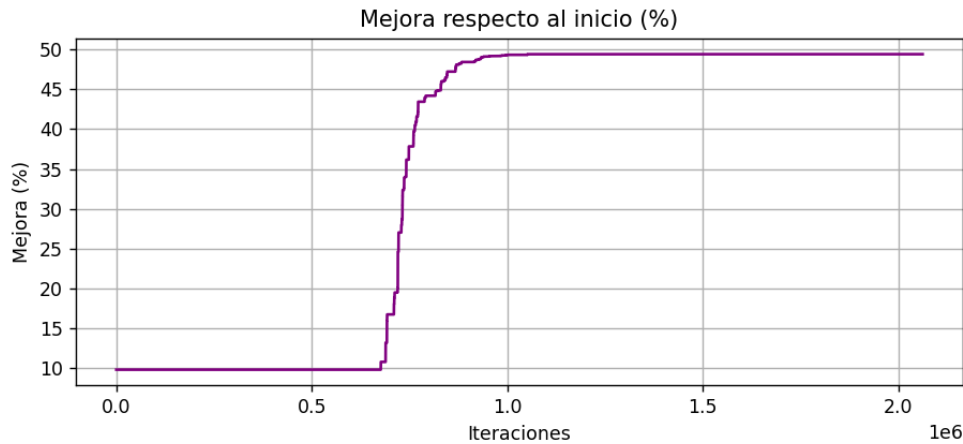
encontrar progresivamente rutas más eficientes hasta estabilizarse en una solución de bajo costo.



Esta gráfica aísla el factor más importante del costo (70%). Se puede observar una clara tendencia a la baja, lo que indica que el algoritmo optimizó las rutas para reducir significativamente el consumo total de combustible, logrando estabilizarse en un mínimo.



Esta gráfica muestra la reducción en los kilómetros totales recorridos. Al igual que las otras métricas, la distancia total disminuye a medida que el algoritmo refina las rutas. Esto confirma que las rutas optimizadas no solo ahorran combustible, sino que también son considerablemente más cortas.



Esta gráfica es el resumen del éxito de la optimización. Demuestra que la solución final es casi un 50% mejor que la solución inicial. El salto abrupto en la gráfica muestra el momento clave en que el algoritmo encontró una configuración de rutas significativamente superior.

7. Agenda de Trabajo

Fecha	Actividad	Descripción	Resultado
01 de octubre	Asignación del proyecto	Recibimos la asignación de optimización de rutas	Comprender el problema y el objetivo
02 – 03 de octubre	Análisis del problema y de los datos	Se revisaron los archivos de Excel que recibimos y su estructura	Datos comprendidos y listos para ser utilizados
04 – 05 de octubre	Desarrollo de la función de Solución Inicial	Se creó la función de solución inicial en donde se asignaron tiendas a cada centro según la distancia	Solución inicial creada
06 – 07 de octubre	Cálculo de costo total y costo de ruta	Se crearon las funciones de costo de ruta y costo total utilizando distancia y el consumo de gasolina	Funciones de cálculo de costos total y de ruta
08-09 de octubre	Desarrollo de la función de Recocido Simulado	Se desarrolló la función del Recocido Simulado con su generación de vecinos y criterio de aceptación	Función de optimización con Recocido Simulado desarrollada
10 de octubre	Pruebas y ajuste de parámetros	Se ajustaron los parámetros del recocido simulado para optimizar los resultados	Resultados optimizados
11 de octubre	Generación del mapa de rutas	Se utilizó folium para visualizar las rutas optimizadas en un mapa	Mapa de las rutas optimizadas generado

12 de octubre	Creación de gráficas y entrega final	Implementación de 4 gráficas utilizando matplotlib para mostrar la evolución del costo, distancia, gasolina y mejora porcentual	Graficas creadas y proyecto finalizado
---------------	--------------------------------------	---	--

8. Conclusiones

En este proyecto, usamos el algoritmo de Recocido Simulado para resolver el problema de cómo repartir productos desde 10 centros de distribución a 90 tiendas en Culiacán, y los resultados fueron excelentes. Logramos que las rutas fueran casi un 50% más eficientes en comparación con la forma más simple de organizarlas, lo cual se puede ver claramente en las gráficas donde los costos bajan muchísimo.

Al final, esto significa que las nuevas rutas son mucho más cortas y gastan bastante menos gasolina. Esto demuestra que el Recocido Simulado es una herramienta muy potente para solucionar problemas de logística del mundo real. El mapa que creamos muestra estas rutas optimizadas de una manera fácil de entender, dándole a la empresa una solución práctica para ahorrar dinero y hacer sus entregas de forma mucho más inteligente.

Referencias

Complexica. (s.f.). *Heuristic Search*. Obtenido de Complexica:

<https://www.complexica.com/narrow-ai-glossary/heuristic-search>

KeepCoding. (s.f.). *¿Qué es la búsqueda heurística y para qué sirve?* Obtenido de KeepCoding.io:

<https://keepcoding.io/blog/que-es-la-busqueda-heuristica-y-para-que-sirve/>

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 671-680.

Laporte, G. (1992). The vehicle routing problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, 59(3), 345-358. Obtenido de [https://doi.org/10.1016/0377-2217\(92\)90192-C](https://doi.org/10.1016/0377-2217(92)90192-C)

Lenstra, J., & Rinnooy Kan, A. (2006). Complexity of vehicle routing and scheduling problems. *Networks*, 221-227.

Osman, I. (1993). Meta-strategy simulated annealing and Tabu search algorithms for the vehicle routine problem. *Annals of Operations Research*, 421-451.

Pearl, J. (1984). *Heuristics: intelligent search strategies for computer problem solving*. Addison-Wesley Longman Publishing Co., Inc.

Thangiah, S. R., Osman, I., & Sun, T. (2014). Hybrid Genetic Algorithm, Simulated Annealing and Tabu Search Methods for Vehicle Routing Problems with Time Windows.